

Docker

Session 4: Introduction to Containerisation & Docker Architecture

1. Overview

This session introduces the core concepts of containerisation and the Docker platform. We begin with the problems that led to the rise of containers, then explore Docker's architecture in depth: how the Docker daemon, client, and registry interact. Internally, we examine Linux primitives that make containers possible—namespaces, control groups, and union filesystems—and clarify the distinction between Docker images and containers. The practical lab walks through installing Docker on Ubuntu and running your first containers.

2. Learning Objectives

- Explain why containers emerged as an alternative to virtual machines for application packaging and deployment.
 - List and describe the components of Docker architecture: client, daemon, and registry.
 - Define Docker images and containers, and explain the layered filesystem using a union mount driver.
 - Understand how Linux namespaces provide isolation and how cgroups enforce resource limits.
 - Install Docker Engine on Ubuntu using the official repository.
 - Run `docker run hello-world` and an Apache HTTP server container.
 - Use essential Docker commands: `docker ps`, `docker images`, `docker pull`, `docker stop`, `docker rm`.
-

3. The Need for Containers

Before containers, production deployments often exposed a painful chasm between development and operations:

- **"Works on my machine" syndrome:** Different library versions, OS configurations, and dependencies between environments.
- **Resource inefficiency:** Virtual machines (VMs) each carry a full guest OS, consuming gigabytes of RAM and taking minutes to boot.
- **Slow scaling:** VM provisioning and configuration are heavyweight processes.

Containers solve these problems by encapsulating an application and its dependencies in a lightweight, portable unit that runs identically anywhere—developer laptop, on-premises server, or cloud. They share the host kernel, start in seconds, and consume only incremental resources.

- **Containerization platforms:** Docker, Podman, containerd, LXC (Linux containers), CRI-O.
-

4. What is Docker?

Docker is an open-source platform that automates the deployment, scaling, and management of applications inside software containers. It provides:

- A standard format for packaging applications (**images**).
- A runtime for executing those packages (**containers**).
- Tooling to build, share, and orchestrate them.

Under the hood, Docker leverages existing Linux kernel features (namespaces, cgroups, capabilities, seccomp, etc.) to create isolated environments that still share the host kernel.

5. Docker Architecture

Docker uses a client-server architecture:



5.1 Docker Daemon (`dockerd`) The daemon runs on the host machine and handles all container-related operations. It listens for Docker API requests (unix socket `/var/run/docker.sock` by default, or optionally TCP). It manages:

- Images (pulling, building, storing)
- Containers (creation, start, stop, deletion)
- Networks and volumes

5.2 Docker Client (`docker`) The command-line tool that sends requests to the daemon via the REST API. Each `docker` command is translated into API calls. The client can talk to a local daemon or a remote engine.

5.3 Docker Registry A registry stores and distributes Docker images. The default public registry is Docker Hub (`hub.docker.com`). Registries can be private (e.g., self-hosted registry, Amazon ECR, GitLab Container Registry). When you run `docker pull nginx`, the daemon contacts the configured registry, downloads the image layers, and stores them locally.

6. Images vs Containers

Understanding the difference is crucial:

Image	Container
A read-only template composed of multiple layers.	A writeable instance of an image.
Built from a Dockerfile; stored immutably.	Created from an image plus a thin, writeable layer on top.
Can be shared via registries.	Ephemeral; can be started, stopped, and destroyed.

Analogy: An image is like an executable program on disk; a container is like a running process. You can run many containers from the same image, each with its own writable layer.

7. Docker Internals – How Isolation Works?

Docker relies on several Linux kernel features to create containers.

7.1 Linux Namespaces Namespaces limit what a process can **see**. Each container gets its own set of namespaces, creating the illusion of a dedicated OS instance.

Namespace	Isolates
PID	Process ID numbers. Container sees only its own processes, starting at PID 1.
NET	Network interfaces, routing tables, firewall rules. Each container can have its own IP address.
IPC	Inter-process communication resources (shared memory, semaphores).
MNT	Mount points. A container sees its own root filesystem.
UTS	Hostname and domain name. Container can have a different hostname than the host.
USER	User and group IDs. A non-root user inside the container can map to a different UID on the host.
CGROUP	Control group view (though cgroups themselves are a separate mechanism).

When a container starts, the `clone()` syscall with the appropriate flags creates new namespaces for the initial process. `unshare` and `setns` can also manipulate namespaces.

7.2 Control Groups (cgroups) Cgroups limit, account for, and isolate resource usage (CPU, memory, disk I/O, network) of process groups. Docker uses cgroups to enforce:

- Memory limits (`--memory`)
- CPU shares (`--cpus`, `--cpuset-cpus`)
- Block I/O weight (`--blkio-weight`)

Without cgroups, a container could consume all host resources, starving other containers and host processes.

7.3 Union Filesystem (Union Mount) Docker images and containers use a union filesystem (Overlay2 is the default on Ubuntu) that stacks layers:

- Multiple read-only **image layers** are mounted together.
- A thin writeable **container layer** sits on top.
- When a file is modified, the union filesystem copies it up (copy-on-write) to the container layer.
- This allows sharing of base image layers across multiple containers, saving disk space and speeding up deployment.

Inspecting layers:

```
docker image inspect nginx:latest | jq .[].RootFS.Layers
```

8. Docker Edition and Installation on Ubuntu

As of today, Docker CE (Community Edition) is the standard free version. We will install from the official Docker repository to ensure the latest stable version.

8.1 Remove old versions

```
sudo apt-get remove docker docker-engine docker.io containerd runc
```

8.2 Set up the repository

```
sudo apt-get update
sudo apt-get install ca-certificates curl gnupg lsb-release -y

# Add Docker's official GPG key
sudo mkdir -m 0755 -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg

# Add the repository
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list >
/dev/null
```

8.3 Install Docker Engine

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin -y
```

8.4 Verify installation

```
sudo docker --version
# Example output: Docker version 29.4.3, build 055a478
```

8.5 Manage Docker as a non-root user

```
sudo groupadd docker          # group likely already exists
sudo usermod -aG docker $USER # add current user in docker group
exit                          # close current terminal
```

```
# in new terminal, user group change will affect
docker run hello-world          # should work without sudo now
```

The `hello-world` container prints a message and exits, confirming the daemon is working correctly.

9. Essential Docker Commands

9.1 Run the Hello-World Container

```
docker run hello-world
```

Observe: The daemon pulled the image from Docker Hub, created a container, and executed it.

9.2 Pull and Run an Apache HTTP Server (httpd)

```
docker pull httpd:alpine      # lightweight Alpine-based image
docker images                 # list images; notice repository, tag, size
```

Run the container in detached mode:

```
docker run -d --name my-apache -p 8080:80 httpd:alpine
```

Breakdown:

- `-d`: detached (runs in background)
- `--name my-apache`: assign a friendly name
- `-p 8080:80`: map host port 8080 to container port 80

Verify the web server is reachable:

```
curl http://localhost:8080
# returns the default Apache "It works!" page
```

You can also open `http://localhost:8080` in a browser.

9.3 Basic Container Lifecycle Commands

```
docker ps                # list running containers
docker ps -a             # show all containers (including stopped)
docker stop my-apache    # gracefully stop the container
docker start my-apache   # restart it
docker logs my-apache    # view container logs
docker inspect my-apache # JSON metadata about the container
docker rm my-apache      # remove the container (must be stopped first or use -f)
```

To clean up after the lab:

```
docker stop my-apache
docker rm my-apache
docker rmi httpd:alpine    # remove the image (if no other containers use it)
```

To clean up all images and containers (use with caution):

```
docker system prune -a --volumes

# Removes:
#   stopped containers
#   unused images
#   unused networks
#   build cache
#   volumes
```

9.4 Inspect Container Internals While the Apache container runs, open an interactive shell inside it:

```
docker exec -it my-apache sh
```

Inside, run `ps aux`, `hostname`, `ip addr`, and `df -h`. Notice:

- The process list shows only container processes (PID namespace).
- The hostname is the container ID or custom name (UTS namespace).
- The network stack shows a container-specific Ethernet interface.
- The root filesystem is the combined image layers + writable layer. Exit the shell with `exit` (the container continues running).

9.5 Quick Look at Underlying Namespaces (Optional) Find the container's PID on the host:

```
docker inspect --format '{{ .State.Pid }}' my-apache
```

Then, list the namespaces of that process:

```
sudo ls -la /proc/<PID>/ns/
```

Each namespace file corresponds to an identifier. You can compare it with the host's namespaces to see isolation.

10. Summary

- Containers package applications with their dependencies into lightweight, portable units that run consistently across environments.
 - Docker's architecture comprises a daemon, a client, and registries; the daemon handles all container management.
 - Images are read-only templates built from multiple layers; containers are running instances with an additional writable layer.
 - Linux namespaces provide isolation of processes, network, mounts, and more, while cgroups restrict resource usage.
 - Installation on Ubuntu involves adding Docker's official repository and installing `docker-ce`.
 - With basic Docker commands, you can pull images, run containers in detached mode, expose ports, and inspect containers at runtime.
-

Session 5: Creating Docker Images Using Dockerfile

1. Overview

Previous part introduced running pre-built images. Now you will learn to build your own. This session covers the Dockerfile: its instructions, how they create image layers, and how the build cache can dramatically accelerate rebuilds. The practical lab guides you through writing production-ready, multi-stage Dockerfiles for the MERN application—one for the Node.js/Express backend and one for the React frontend. The resulting images are then tagged and pushed to Docker Hub, making them available for deployment anywhere.

2. Learning Objectives

- Explain the purpose and structure of a Dockerfile as the declarative blueprint for an image.
 - Describe every core Dockerfile instruction, its syntax (shell vs. exec form), and its effect on the image filesystem and container behaviour.
 - Understand the copy-on-write layered filesystem and how Docker's build cache works to skip unchanged steps.
 - Apply best practices for layer ordering, dependency separation, and the use of `.dockerignore`.
 - Construct multi-stage Dockerfiles that separate build tools from the final runtime, producing minimal, secure images.
 - Build, tag, and push images to Docker Hub from the command line.
-

3. What Is a Dockerfile?

A Dockerfile is a plain-text file named `Dockerfile` (no extension) that contains a sequence of instructions. Each instruction creates a new filesystem layer in the final image. The `docker build` command reads the Dockerfile and the surrounding directory (the **build context**) to produce an image.

The build context is everything in the directory where you run `docker build`. Docker sends this entire context to the daemon unless you exclude files with a `.dockerignore` file. Keeping the context lean speeds up builds.

4. Dockerfile Instructions – A Complete Reference

4.1 FROM

```
FROM <image>[:<tag>] [AS <stage-name>]
```

Sets the base image for the current build stage. Every Dockerfile must begin with **FROM** (except **ARG**, which may appear before it). In multi-stage builds, you can name a stage with **AS** and later reference it with **COPY --from=<stage>**.

4.2 LABEL

```
LABEL maintainer="team@example.com"  
LABEL version="1.2.0" description="MERN backend"
```

Adds arbitrary key-value metadata to the image. Multiple labels can be set in a single instruction to avoid creating extra layers.

4.3 ENV

```
ENV NODE_ENV=production  
ENV PORT=5000
```

Sets environment variables that persist both at build time and at container runtime. Prefer **ENV** for values that should be available inside the running container.

4.4 ARG

```
ARG NODE_VERSION=22  
FROM node:${NODE_VERSION}-alpine
```

Defines a build-time variable. Unlike **ENV**, **ARG** values are not present in the final image unless explicitly redeclared. They are ideal for version pinning that may change per build.

4.5 WORKDIR

```
WORKDIR /app
```

Sets the working directory for all subsequent instructions (**RUN**, **CMD**, **ENTRYPOINT**, **COPY**, **ADD**). If the directory does not exist, it is created. Using **WORKDIR** is preferred over chaining **RUN mkdir -p && cd**.

4.6 COPY

```
COPY backend/package*.json ./
COPY backend/ ./
COPY --from=builder /app/build /usr/share/nginx/html
```

Copies files and directories from the build context (or from another stage) into the image filesystem. **COPY** is preferred over **ADD** because it is explicit and predictable. **ADD** has extra features (remote URL download, tar extraction) that are rarely needed.

4.7 RUN

```
RUN npm ci --only=production
```

Executes commands inside the image during build. Each **RUN** creates a new layer. Chain commands with **&&** to minimise layer count:

```
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
```

Contrast with **CMD** and **ENTRYPOINT**, which set the command that runs when the container starts.

4.8 EXPOSE

```
EXPOSE 5000
```

Documents the port the container listens on. It does **not** actually publish the port. Publishing requires **-p** with **docker run** or the **ports** section in a Compose file.

4.9 ENTRYPOINT

```
ENTRYPOINT ["docker-entrypoint.sh"]
```

Defines the executable that must always run when the container starts. Unlike **CMD**, runtime arguments to **docker run** are appended to the entrypoint rather than replacing it. This makes **ENTRYPOINT** suitable for wrapper scripts and fixed-purpose containers. The exec form (JSON array) is recommended because it runs the binary directly as PID 1, enabling proper signal handling. The shell form wraps the command in **/bin/sh -c**, which intercepts UNIX signals (SIGTERM) and may cause the container to ignore a **docker stop** request until the timeout expires.

4.10 CMD

```
CMD ["node", "server.js"]           # exec form (preferred)
```

```
CMD node server.js # shell form
```

Provides a default command (with arguments) for the container. If there are multiple **CMD** instructions, only the last one takes effect. Like **ENTRYPOINT**, the exec form is preferred for signal handling.

When combined with **ENTRYPOINT**, **CMD** supplies default arguments to the entrypoint executable.

```
ENTRYPOINT ["python", "app.py"]  
CMD ["--help"]
```

- **docker run my-app** executes: `python app.py --help`
- **docker run my-app --version** executes: `python app.py --version`

4.11 USER

```
USER node
```

Switches to the specified user for all subsequent **RUN**, **CMD**, and **ENTRYPOINT** instructions. Running containers as a non-root user is a fundamental security practice.

4.12 VOLUME

```
VOLUME /data
```

Creates a mount point and marks it as an externally mounted volume. Prefer using **docker run -v** or Compose **volumes:** for explicit control.

4.13 HEALTHCHECK

```
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \  
CMD curl -f http://localhost:5000/health || exit 1
```

Tells Docker how to test whether the container is healthy. Without a healthcheck, Docker cannot detect a container that is running but malfunctioning.

5. Layers and the Build Cache

Each **FROM**, **RUN**, **COPY**, and **ADD** instruction creates a new read-only layer in the image. Layers are stacked using an Overlay2 union filesystem. During a build, Docker checks whether an instruction and its inputs are identical to a previous build. If they are, the cached layer is reused and the instruction is skipped. This cache check is performed sequentially: as soon as one layer's cache is invalidated, all subsequent layers are rebuilt, even if their individual inputs have not changed.

Therefore, ordering Dockerfile instructions with the least-frequently-changing items first is the single most effective optimisation.

```
# Avoid:
COPY . .          # any file change invalidates cache
RUN npm ci        # dependency install always re-runs

# Prefer:
COPY package*.json ./
RUN npm ci        # cached unless package*.json changes
COPY . .          # only source code copies are uncached
```

This pattern ensures that dependency installation—often the most time-consuming step—is cached and only re-executed when dependencies actually change.

6. Multi-Stage Builds

A multi-stage build uses more than one **FROM** instruction in the same Dockerfile. Each **FROM** begins a new stage with a clean base image. Stages can selectively copy artifacts from earlier stages using **COPY --from=<stage>**.

Benefits:

- **Separation of concerns:** Build tools (compilers, npm, test frameworks) live only in builder stages.
- **Smaller final images:** The runtime image contains only the compiled output, not the entire toolchain.
- **Better security:** Reduced attack surface; build-time secrets and tools are absent from production.

Example:

```
Stage 1 (builder): FROM node:22-alpine → install deps → compile → produce static files
Stage 2 (runner):  FROM nginx:alpine → COPY only the static files from Stage 1
```

The final image is a bare Nginx server with static assets—no Node.js runtime, no npm, no source code. For a React app, this can reduce image size from approximately 760 MB to under 100 MB.

7. The **.dockerignore** File

Before a build, Docker reads a **.dockerignore** file in the build context root. Patterns matching files and directories are excluded from the context. This:

- Shrinks the context size, reducing the initial upload time.
- Prevents sensitive files (.env, keys, credentials) from being accidentally copied.
- Stops large, irrelevant directories (node_modules, .git) from entering the image.

```
node_modules
build
.git
.gitignore
*.md
.env
```

8. Practical Lab – Building and Pushing the MERN Images

Final output

```
mern-app/
├── backend/
│   ├── server.js
│   ├── package.json
│   └── Dockerfile
├── frontend/
│   ├── src/
│   ├── public/
│   ├── package.json
│   ├── vite.config.js
│   ├── nginx.conf
│   └── Dockerfile
└── .dockerignore
```

1. Create Project Folder

```
mkdir mern-app
cd mern-app
```

2. Create the Backend (Express)

Create backend folder:

```
mkdir backend
```

2.1 Create `server.js`

```
touch backend/server.js
```

Add this code:

```
const express = require("express");

const app = express();

const PORT = process.env.PORT || 5000;

app.get("/", (req, res) => {
  res.send("Backend is running...");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

2.2 Create package.json

Create:

```
touch backend/package.json
```

Add:

```
{
  "name": "backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

3. Create the Frontend (Vite)

Inside project root:

```
npm create vite@latest frontend -- --template react
```

Go inside frontend:

```
cd frontend
npm install
cd ..
```

4. Test Apps Locally First

Before Docker, ALWAYS verify apps work normally.

Backend

Install backend dependencies:

```
cd backend
npm install
npm start
```

Open:

```
http://localhost:5000
```

You should see:

```
Backend is running...
```

Stop server with:

```
CTRL + C
```

Frontend

Run Vite:

```
cd frontend
```

```
npm run dev
```

Open:

```
http://localhost:5173
```

You should see the React app.

Stop with:

```
CTRL + C
```

5. Create Backend Dockerfile

Create:

```
touch backend/Dockerfile
```

Add:

```
FROM node:22-alpine
WORKDIR /app
COPY backend/package*.json ./
RUN npm ci
COPY backend .
EXPOSE 5000
CMD ["npm", "start"]
```

Understanding This Dockerfile

- FROM

```
FROM node:22-alpine
```

Uses a lightweight Node.js Linux image.

- WORKDIR

```
WORKDIR /app
```

All commands now run inside `/app`.

- COPY

```
COPY backend/package*.json ./
```

Copies package files first.

This improves Docker cache efficiency.

- RUN

```
RUN npm ci
```

Installs dependencies inside image.

- Second COPY

```
COPY backend .
```

Copies backend source code.

- EXPOSE

```
## Create `frontend/Dockerfile`  
dockerfile  
EXPOSE 5000
```

Documents container port.

- CMD

```
CMD ["node", "server.js"]
```

Starts app when container runs.

6. Create Frontend Dockerfile (Vite + Nginx)

- Vite builds static files.
- Nginx serves them efficiently.

Create `frontend/Dockerfile`

```
touch frontend/Dockerfile
```

Add:

```
# Build stage
FROM node:22-alpine AS builder
WORKDIR /app
COPY frontend/package*.json ./
RUN npm ci
COPY frontend .
RUN npm run build

# Production stage
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY frontend/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

7. Create Nginx Config

Create:

```
touch frontend/nginx.conf
```

Add:

```
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html;

        try_files $uri $uri/ /index.html;
    }
}
```

- Why `try_files` Matters?
 - React Router uses client-side routing.
 - Without this:

```
/dashboard  
/settings  
/profile
```

- would return:

```
404 Not Found
```

- after refresh.
- `try_files` fixes it.

8. Create `.dockerignore`

In project root:

```
touch .dockerignore
```

Add:

```
node_modules  
.git  
frontend/node_modules  
backend/node_modules  
frontend/dist
```

- Why `.dockerignore` Is Important? Without it:
 - Docker sends huge folders to daemon
 - Builds become slow
 - Image sizes increase

9. Build Backend Image

From project root:

```
docker build -t mern-backend -f backend/Dockerfile .
```

10. Run Backend Container

```
docker run -d --name backend-container -p 5000:5000 mern-backend
```

- and Test Backend

```
curl http://localhost:5000
```

- Expected:

Backend is running...

View Logs

```
docker logs backend-container
```

Stop Container

```
docker stop backend-container  
docker rm backend-container
```

11. Build Frontend Image

```
docker build -t mern-frontend -f frontend/Dockerfile .
```

12. Run Frontend Container

```
docker run -d --name frontend-container -p 8080:80 mern-frontend
```

13. Open Frontend

Visit:

```
http://localhost:8080
```

You should see your Vite React app.

Multi-Stage Build

Frontend Dockerfile uses:

```
FROM node AS builder
```

then:

```
FROM nginx
```

- Result:
 - smaller image
 - faster startup
 - production optimized
- Note: The single-stage image will be several times larger than the multi-stage version. This is because it retains Node.js, npm, and all build-time dependencies, none of which are needed to serve static files at runtime.

15. Useful Docker Commands

```
## Running Containers
docker ps

## All Containers
docker ps -a

## Images
docker images

## Remove Container
docker rm <container-id>

## Remove Image
docker rmi <image-id>

## View Logs
docker logs <container-name>
```

9. Practical Lab – Using DockerHub

1. Create/login to Docker Hub

2. Tag images properly
3. Push images
4. Pull them back to verify
5. Understand image naming conventions

1. Create a Docker Hub Account

- Go to: <https://hub.docker.com>
- Create a free account if you don't already have one.
- Your username matters because image names include it. Example:

```
yourusername/mern-backend
```

- In settings, generate access token. This is to be used later for login.

2. Login from Terminal

```
docker login
```

- You'll see:

```
Username:  
Password: (use token here)
```

- After successful login:

```
Login Succeeded
```

3. Understand Docker Image Naming

Right now your image is probably:

```
mern-backend
```

- But Docker Hub requires:

```
<dockerhub-username>/<repository>:<tag>
```

- Example:

```
john/mern-backend:1.0
```

4. Tag Backend Image

- Replace **YOUR_USERNAME**.

```
docker tag mern-backend YOUR_USERNAME/mern-backend:1.0
```

- Verify:

```
docker images
```

- You'll now see BOTH:

```
mern-backend  
YOUR_USERNAME/mern-backend
```

- This does NOT duplicate the image. It just adds another label.
- You can note image ids.

5. Tag Frontend Image

```
docker tag mern-frontend YOUR_USERNAME/mern-frontend:1.0
```

6. Push Backend Image

```
docker push YOUR_USERNAME/mern-backend:1.0
```

- First push may take some time.
- You'll see layers uploading.
- Eventually:

```
latest: digest: sha256:...
```

7. Push Frontend Image

```
docker push YOUR_USERNAME/mern-frontend:1.0
```

8. Verify on Docker Hub

- Open:

```
https://hub.docker.com/repositories
```

- You should now see:
 - `mern-backend`
 - `mern-frontend`

9. Test Pulling Images

- Delete local image first:

```
docker rmi YOUR_USERNAME/mern-backend:1.0
```

- Now pull from Docker Hub:

```
docker pull YOUR_USERNAME/mern-backend:1.0
```

10. Run Pulled Image

```
docker run -p 5000:5000 YOUR_USERNAME/mern-backend:1.0
```

- Open:

```
http://localhost:5000
```

- If it works:
 - Your image is portable
 - Anyone can run it
 - Deployment becomes easy

11. Why Docker Hub?

Docker Hub is basically: GitHub for Docker images

- You can:
 - share apps
 - deploy to servers
 - use in Kubernetes
 - use in CI/CD pipelines
 - avoid rebuilding everywhere
-

10. Summary

- A Dockerfile is a sequence of instructions that define how to assemble an image. Each instruction typically creates one filesystem layer.
 - Key instructions include `FROM`, `RUN`, `COPY`, `ENV`, `WORKDIR`, `EXPOSE`, `CMD`, and `ENTRYPOINT`. Understanding their semantics—especially shell vs. exec form—is essential for reliable container behaviour.
 - Docker's layer caching mechanism reuses unchanged layers, dramatically shortening rebuilds. Ordering instructions by change frequency is the primary optimisation technique.
 - Multi-stage builds allow you to compile in a heavyweight builder stage and copy only the necessary runtime artifacts into a slim final image, reducing both size and attack surface.
 - A `.dockerignore` file prevents unnecessary or sensitive files from entering the build context.
 - The practical lab produced production-ready images for a MERN backend and a React frontend, tagged and pushed them to Docker Hub, and demonstrated the size advantage of multi-stage builds.
-

Session 6: Container Lifecycle & Management

1. Overview

Running `docker run` is only the beginning of container management. This session provides a complete understanding of the container lifecycle — from creation to removal — and equips you with essential commands for debugging, maintaining, and persisting data. You will learn the precise meaning of each lifecycle state, how to inspect and interact with running containers, how port mapping works, and when to use volumes versus bind mounts. The practical lab uses the MERN images built in Session 5 and demonstrates real-world operations: debugging with `docker exec`, tailing logs, copying files, and cleaning up resources.

2. Learning Objectives

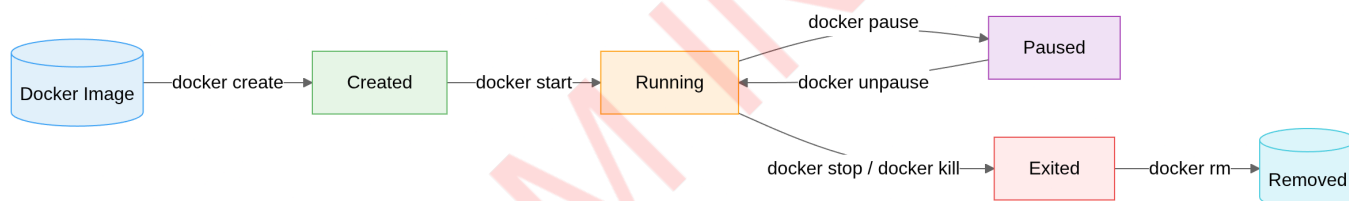
- List and describe all container lifecycle states and the commands that transition between them.
- Execute commands inside a running container using `docker exec` and distinguish between interactive and non-interactive modes.
- Retrieve and interpret container logs with various formatting options.
- Explain the difference between exposed ports, published ports, and endpoint-specific bindings.
- Compare Docker volumes and bind mounts, and select the appropriate persistence mechanism for a given scenario.
- Copy files between the host and a container using `docker cp`.
- Monitor container resource usage with `docker stats` and `docker top`.
- Perform systematic cleanup of containers, images, and volumes.

3. Container Lifecycle States

A Docker container progresses through distinct states. Understanding these states helps diagnose issues and write reliable automation.

State	Description
created	The container has been created (via <code>docker create</code>) but never started. The filesystem and configuration exist but no process runs.
running	The container's main process (PID 1) is executing. This state is entered after <code>docker start</code> or <code>docker run</code> .
paused	The container's processes are suspended (via <code>docker pause</code>). All cgroups are frozen; the process memory remains but CPU time stops. Use <code>docker unpause</code> to resume.
exited	The container's main process has terminated. The exit code indicates success (0) or failure (non-zero). This state is often called "stopped".
dead	The container is partially removed. This is a transitional state you rarely encounter; it occurs if the daemon fails to fully remove a container.

Lifecycle transitions:



- `docker run` is equivalent to `docker create` followed by `docker start`.
- `docker stop` sends SIGTERM to PID 1, then after a grace period (default 10 s) SIGKILL. `docker kill` sends a signal immediately (default SIGKILL).
- `docker pause` uses the cgroups freezer subsystem to suspend the process without terminating it.

To see all containers including exited ones: `docker ps -a`.

4. Core Container Management Commands

4.1 Starting, Stopping, and Restarting

```
docker start my-container      # from created or exited state to running
docker stop my-container       # graceful shutdown (SIGTERM → SIGKILL)
docker restart my-container    # stop + start
docker pause my-container      # freeze
docker unpause my-container     # resume
docker kill --signal=SIGINT my-container  # send custom signal
```

Use `docker stop -t 30` to increase the grace period before SIGKILL for applications that need longer to finish in-flight requests.

4.2 Removing Containers

```
docker rm my-container          # remove an exited container
docker rm -f my-container       # force remove a running container (SIGKILL then
remove)
docker container prune          # remove all stopped containers
```

4.3 Executing Commands Inside a Container

```
docker exec my-container ls -la /app
docker exec -it my-container sh  # interactive shell (tty)
docker exec -u node my-container whoami
docker exec -e DEBUG=true my-container node script.js
```

- Without `-it`, the command runs non-interactively and `stdio` is not connected to a terminal.
- The `exec` process runs in the same namespaces as the container's main process; it sees the same filesystem, process list (with the main process), and network interfaces.
- The command runs as a child of the runtime `containerd` shim, not as a child of the container's PID 1. If PID 1 dies, `exec` sessions are terminated.

4.4 Attaching to a Container

```
docker attach my-container
```

- Connects to the container's PID 1 process's `stdin/stdout/stderr`.
- Exiting the attach session (Ctrl+C) typically kills PID 1 because the signal is forwarded. Use `Ctrl+P`, `Ctrl+Q` to detach without stopping the container.
- `docker exec` is generally preferred because it creates a new process, leaving the main process undisturbed.

5. Logging and Inspection

5.1 Viewing Logs

```
docker logs my-container
docker logs --tail 50 my-container    # last 50 lines
docker logs -f my-container           # follow (tail -f)
docker logs --since 10m my-container  # logs from the last 10 minutes
docker logs --timestamps my-container # include timestamps
```

Docker captures stdout and stderr of PID 1. The default logging driver is `json-file`, which stores logs on disk at `/var/lib/docker/containers/<id>/<id>-json.log`. In production, you may configure a different logging driver (syslog, journald, fluentd) or use a log shipper.

5.2 Inspecting Container Metadata

```
docker inspect my-container
docker inspect --format '{{.State.Status}} {{.NetworkSettings.IPAddress}}' my-
container
```

`docker inspect` returns a JSON object with every detail: all mount points, environment variables, exposed ports, network settings, and more. The Go-template syntax enables extracting specific fields.

5.3 Viewing Processes Inside a Container

```
docker top my-container
```

This provides a `ps`-like view of processes running inside the container's PID namespace, including their PID on the host and the PID inside the container.

6. Port Mapping

When a container runs, it has a private network namespace with its own loopback and Ethernet interfaces. To reach a service inside the container from the host or external network, ports must be published.

Syntax:

```
-p <host-port>:<container-port>
-p <host-ip>:<host-port>:<container-port>
```

Examples:

- `-p 8080:80`: bind host port 8080 to container port 80 on all interfaces.
- `-p 127.0.0.1:8080:80`: bind only to loopback, not reachable externally.
- `-p 80`: allocate a random host port and map it to container port 80.
- `-P`: publish all ports declared with `EXPOSE` to random high-numbered ports.

Under the hood, Docker manipulates iptables NAT rules in the `PREROUTING` and `OUTPUT` chains (when `docker-proxy` is off) or runs a userland proxy process. If in your docker version, the userland proxy is disabled, it relies solely on iptables for performance.

To list port mappings of a running container:

```
docker port my-container
```

7. Volumes and Bind Mounts

Containers are ephemeral; their writable layer is discarded when the container is removed. To persist data, you must use volumes or bind mounts.

Feature	Volume	Bind Mount
Management	Managed by Docker (<code>docker volume create</code>)	Host filesystem path
Storage location	<code>/var/lib/docker/volumes/</code> (on Linux)	Any absolute host path
Populated from image?	Yes, on first use if the volume is empty	No, mount covers existing content
Portability	High, can be named and shared among containers	Low, tied to host directory structure
Use case	Databases, application data, stateful sets	Development hot-reload, configuration files

Syntax:

```
# Volume: docker manages the location
-v my-volume:/app/data
--mount source=my-volume,target=/app/data

# Bind mount: host path to container path
-v /home/user/app/config:/app/config
--mount type=bind,source=/home/user/app/config,target=/app/config
```

The `--mount` form is more explicit and is required for some advanced options (read-only, volume options). The shorter `-v` form will create a volume if the source does not already exist as a directory (for bind mounts) or as a volume name. With `--mount`, you must specify `type=bind` or `type=volume`.

Bind Mount for Development: Bind mounts connect a host directory directly into the container. This is heavily used in local development to reflect source code changes instantly without rebuilding the image. For example, if you mount the host's `frontend/build` directory over the container's `/usr/share/nginx/html`, any React rebuild on the host immediately updates the web server.

8. Copying Files Between Host and Container

```
docker cp <host-path> <container>:<container-path>
docker cp <container>:<container-path> <host-path>
```

- Works for both running and stopped containers.
- Copies file ownership and permissions as they appear inside the container.
- For large or many files, `docker cp` creates a tar stream internally.

While `docker cp` is useful for one-off tasks (e.g., extracting a core dump, dropping a configuration file into a stopped container), it should not be used as a primary deployment mechanism. Reproducible deployments use images and, when runtime configuration is needed, environment variables, volumes, or secrets.

9. Practical Lab

9.1 Environment Setup Ensure you have the MERN images from Session 5 available:

```
docker images | grep mern
```

If not present, pull from Docker Hub or rebuild from the project directory.

9.2 Lifecycle Demonstration

```
# Start a backend container
docker run -d --name lifecycle-demo -p 5001:5000 mern-backend:1.0.0
docker ps                # state: running
curl http://localhost:5001/

# Pause and unpause
docker pause lifecycle-demo
curl http://localhost:5001/  # will hang (process frozen)
docker unpause lifecycle-demo
curl http://localhost:5001/  # responds again

# Stop and verify exited state
docker stop lifecycle-demo
docker ps -a | grep lifecycle-demo  # Exited (0)

# Start again
docker start lifecycle-demo
docker ps                # running again

# Kill and remove
docker kill lifecycle-demo
docker rm lifecycle-demo
```

9.3 Executing Commands Inside the Container

```
docker run -d --name debug-backend -p 5002:5000 mern-backend:1.0.0
docker exec debug-backend ls -la /app
docker exec debug-backend env | sort
docker exec debug-backend cat /etc/os-release
docker exec -it debug-backend sh
# Inside container: ps aux, hostname, id, exit
```

Use `docker exec` to run `npm test` or any custom script inside the container without a new build.

9.4 Log Inspection

```
docker logs debug-backend          # entire output so far
docker logs --tail 20 debug-backend # last 20 lines
docker logs -f --since 1m debug-backend
# Send a few requests in another terminal
curl http://localhost:5002/
docker logs --timestamps debug-backend | grep GET
```

9.5 Using `docker cp`

```
# Create a temporary file on the host
echo "console.log('Copied from host');" > /tmp/test.js
docker cp /tmp/test.js debug-backend:/tmp/test.js
docker exec debug-backend cat /tmp/test.js

# Copy a file from container to host
docker cp debug-backend:/app/server.js /tmp/server-from-container.js
head /tmp/server-from-container.js
```

Demonstrate that the copy is successful but note that restarting the container does not apply the copied file as a code change; the original image layer still takes precedence unless the writable layer file overrides it (which is unreliable). This reinforces why images or volumes are the correct way to deploy code.

9.6 Bind Mount for Live Configuration While the frontend container is running, we can override the Nginx configuration via a bind mount without rebuilding the image.

Create a custom HTML file:

```
mkdir -p ~/devops-course/lab/mounts
echo "<h1>Hello from Bind Mount</h1>" > ~/devops-course/lab/mounts/index.html
```

Run the frontend container with a bind mount to replace `index.html`:

```
docker run -d --name bindmount-frontend -p 8081:80 \
  -v ~/devops-course/lab/mounts/index.html:/usr/share/nginx/html/index.html:ro \
  mern-frontend:1.0.0
curl http://localhost:8081/
```

You should see "Hello from Bind Mount" instead of the default React app. The `:ro` makes the mount read-only, protecting the container from accidentally modifying the host file.

Now mount the entire build output directory from host to container for development:

```
# Suppose we build the React app on the host
cd ~/devops-course/mern-app/frontend
npm run build
docker run -d --name dev-frontend -p 8082:80 \
  -v ~/devops-course/mern-app/frontend/build:/usr/share/nginx/html:ro \
  mern-frontend:1.0.0
```

Any host rebuild updates the container immediately without restart (Nginx serves static files afresh on each request).

9.7 Volumes for Persistent Data Create a named volume and use it with the backend:

```
docker volume create backend-uploads
docker run -d --name volume-backend -p 5003:5000 \
  -v backend-uploads:/app/uploads \
  mern-backend:1.0.0
docker exec volume-backend touch /app/uploads/test.txt
docker stop volume-backend && docker rm volume-backend
docker run -d --name volume-backend2 -p 5004:5000 \
  -v backend-uploads:/app/uploads \
  mern-backend:1.0.0
docker exec volume-backend2 ls /app/uploads # test.txt still exists
```

The data persisted across container replacement because the volume lifecycle is independent.

9.8 Container Resource Monitoring

```
docker stats --no-stream debug-backend
docker top debug-backend
```

For a real-time view:

```
docker stats
```

Press Ctrl+C to exit. This shows CPU%, memory usage, and network I/O per container.

9.9 Cleanup

```
docker stop debug-backend bindmount-frontend dev-frontend volume-backend2
docker rm debug-backend bindmount-frontend dev-frontend volume-backend2

# Remove the named volume (optional)
docker volume rm backend-uploads

# Prune all unused data (containers, networks, images, volumes)
docker system prune -a
```

The `-a` flag removes all unused images, not just dangling layers.

10. Summary

- A container moves through discrete states: created, running, paused, exited (stopped), and dead. Each state is managed via `create`, `start`, `stop`, `pause`, `unpause`, `kill`, and `rm`.
- `docker exec` creates a new process inside the container's namespaces, ideal for debugging and administration without affecting the main application.
- Port mapping (`-p`) exposes container services to the host's network. Without it, containers are reachable only within their private Docker network.
- Two persistence mechanisms exist: volumes (Docker-managed) and bind mounts (host path). Volumes are the recommended way to persist application data; bind mounts are excellent for development and configuration overrides.
- `docker cp` is a handy but limited tool for transferring files; it does not replace proper image management or volume mounting.
- Monitoring tools like `docker logs`, `docker top`, and `docker stats` provide visibility into container behaviour and resource consumption.
- Regular cleanup with `docker container prune` and `docker image prune` prevents disk exhaustion on build servers.

Session 7: Introduction to YAML

1. Overview

YAML has become the lingua franca of configuration in the DevOps ecosystem. It is used to define everything from Docker Compose stacks and Kubernetes manifests to Ansible playbooks and CI/CD pipeline definitions (GitHub Actions, GitLab CI when using declarative syntax). This session provides a thorough grounding in YAML syntax, its data model, and advanced features like anchors and multi-document files. The practical component includes writing a sample container deployment specification in YAML and validating it with `yamllint`.

2. Learning Objectives

- Explain the purpose of YAML, its design philosophy, and its role in modern infrastructure as code and configuration management.
- Understand YAML's data types: scalars, sequences (lists), and mappings (dictionaries), and the difference between block and flow styles.
- Handle multi-line strings correctly using literal and folded scalars.
- Use anchors and aliases to reduce duplication in YAML files.
- Structure multi-document YAML files with document separators.
- Write a practical YAML file describing a container deployment, environment variables, and volumes.
- Install `yamllint` and validate YAML files to ensure syntactic correctness and adherence to best practices.

3. What Is YAML?

YAML originally (2001) stood for "Yet Another Markup Language" but today (since 2002) it is a recursive acronym for "YAML Ain't Markup Language". It is a human-friendly data serialisation standard for all programming languages. Its goals are:

- **Human readability:** Minimise structural characters. Indentation defines nesting, and data types are inferred from syntax.
- **Expressiveness:** Can represent complex data structures, including references (anchors/aliases) to avoid repetition.
- **Interoperability:** Available in virtually every programming language (Python `PyYAML`, Node.js `js-yaml`, Java `SnakeYAML`, etc.).

YAML is a strict **superset of JSON**: every valid JSON document is also a valid YAML document. However, YAML's native block style is far cleaner for human-authored configuration.

Comparison with JSON and XML:

Feature	YAML	JSON	XML
Readability	Very high	Moderate	Verbose
Comments	#	Not allowed	<!-- ... -->
Native data types	Yes (scalars, lists, maps, dates, etc.)	No (must be inferred)	No (text only)
Indentation	Meaningful	Not meaningful	Not meaningful
Size	Compact	Compact	Large
Anchors & aliases	Built-in	Not standard	XPointer (rarely used)

In DevOps, YAML's ability to comment configuration and its clean representation of hierarchical data have made it the de facto choice.

4. Basic Syntax Rules

- **Indentation:** YAML uses spaces (not tabs) to denote nesting. Two spaces is conventional, but any consistent number of spaces may be used (one is not recommended because it is hard to distinguish from a single-character key). **Tabs are forbidden**; any tab character causes a parse error.
- **Case sensitivity:** Keys are case-sensitive.
- **File extension:** `.yaml` or `.yml`; both are accepted.
- **Comments:** Begin with `#` (including the space after it is conventional). Comments can appear on their own line or after a value.
- **Document start/end:** `---` denotes the start of a document; `...` optionally denotes the end. Without explicit markers, YAML uses a single document per file.

5. Data Types in Depth

5.1 Scalars Scalars are single, indivisible values. YAML inferring types can be convenient but sometimes leads to ambiguity. Common scalars:

Strings

```
string: hello world
quoted_string: "hello: world" # Must quote if value contains special chars like :,
{, }, [, ], ,, &, *, ?, |, -, <, >, =, !, %, @, `
single_quoted: 'This is a string # No escape sequences, just literal '
multi_line_literal: | # Literal block: preserves newlines and indentation
    This is a long
    string with several
    lines.
multi_line_folded:
    > # Folded block: newlines become spaces, blank lines become newline
    This is a long
    string that will be
    folded into a single line.
```

Numbers

```
integer: 42
float: 3.14159
hexadecimal: 0xFF
scientific: 1.2e+3
# To force a number to be a string (e.g., version 1.10), quote it:
version: "1.10"
```

Booleans

```
true_value: true # also: True, TRUE, yes, Yes, on, On, ON
false_value: false # also: False, FALSE, no, No, off, Off, OFF
```

Quoting "yes" or "no" ensures they remain strings.

Null

```
empty_value: null # also: ~, or just empty (value: )
```

5.2 Sequences (Lists) Lists are ordered collections. They can be written in two styles:

Block style:

```
ports:
- 80
- 443
- 8080
```

Items are introduced by a hyphen and a space.

Flow style (compact, JSON-like):

```
ports: [80, 443, 8080]
```

5.3 Mappings (Dictionaries/Associative Arrays) Mappings are collections of key-value pairs.

Block style:

```
server:
  host: localhost
  port: 8080
```

Flow style:

```
server: { host: localhost, port: 8080 }
```

5.4 Complex Nesting Lists can contain maps and maps can contain lists.

```
environments:
  dev:
    name: development
    replicas: 1
    resources:
      limits:
        cpu: "0.5"
```

```

    memory: "256Mi"
    requests:
      cpu: "0.1"
      memory: "128Mi"
  prod:
    name: production
    replicas: 3
    resources:
      limits:
        cpu: "2"
        memory: "2Gi"

```

6. Advanced Features

6.1 Anchors and Aliases Anchors (&) mark a node for later reference. Aliases (*) reference the anchored node. This avoids repeating identical configuration blocks, particularly useful for complex Kubernetes manifests or CI workflows.

Example:

```

default_settings: &defaults
  restart: always
  log_level: info

service_a:
  <<: *defaults # merge key: pulls in all keys from defaults
  name: alpha
  port: 8080

service_b:
  <<: *defaults
  name: beta
  port: 9090
  log_level: debug # override single key

```

The <<: merge key is a YAML type that merges mappings. Not all parsers recognise it, but the standard `yaml` library in Python, Ruby, and `SnakeYAML` support it. To be safe, you can explicitly list keys and reference values individually:

```

default_restart: &restart always
service_a:
  restart: *restart

```

An alias cannot be defined before its anchor. Anchors are scoped to the current document.

6.2 Tags (Advanced) YAML types can be explicitly specified with tags (e.g., `!!str`, `!!int`, `!!timestamp`). In DevOps contexts they are rarely used; relying on quoting is simpler.

6.3 Multi-Document Files A single YAML file can contain several independent documents, separated by `---`. This is common in Kubernetes, where you define multiple resources in one file.

```
---
apiVersion: v1
kind: Namespace
metadata:
  name: staging
---
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: staging
spec:
  ports:
    - port: 80
```

The optional `...` ends a document without starting another.

7. Practical Lab – Writing and Validating a Container Deployment File

7.1 Example: Docker Compose-like YAML (but not yet Compose) Create a file `container-deploy.yaml` that describes a container specification similar to what you will later use in Docker Compose. The file will define a service (our MERN backend), environment variables, ports, and a volume.

```
# container-deploy.yaml
# A specification for deploying the MERN backend container

version: "3.9" # docker-compose version (for future use)

services:
  backend:
    image: mern-backend:1.0.0
    container_name: mern-backend-prod
    restart: always
    ports:
      - "5000:5000"
    environment:
      NODE_ENV: production
      DB_HOST: mysql
      DB_USER: mern_user
      DB_PASS: "${DB_PASSWORD}" # variable from environment or .env
    volumes:
      - backend-uploads:/app/uploads
      - ./custom-config/app.yaml:/app/config/app.yaml:ro
```

```
volumes:
  backend-uploads:
    driver: local
```

Explain each section:

- `services.backend` is a mapping with keys like `image`, `ports`, etc.
- `environment` uses a mapping; values can contain variable interpolation via `$VAR` (shell syntax, Docker Compose feature).
- `volumes` list uses the `- service-volume:/container/path` syntax with optional `:ro` for read-only.
- Top-level `volumes`: declares the named volume.

This is a realistic precursor to the Docker Compose file we will write later. For now, it serves as a complex YAML test.

7.2 Install yamllint Yamllint is a linter for YAML files that checks syntax and style rules.

```
sudo apt update
sudo apt install yamllint -y
```

Verify:

```
yamllint --version
# Should be 1.28.0 or later (as of mid-2020s)
```

7.3 Lint the File Run yamllint on the created file:

```
yamllint container-deploy.yaml
```

If no errors, it returns clean. Introduce a deliberate error:

```
sed -i 's/    ports:/ \tports:/' container-deploy.yaml
yamllint container-deploy.yaml
# Error: tabs used for indentation
```

Fix it by reverting.

Common yamllint rules:

- `indentation`: enforces spaces, consistent width.
- `line-length`: max line length (default 80 chars).
- `trailing-spaces`.

- **truthy**: warns when strings like **yes/no** are unquoted and could be misinterpreted as boolean.

You can configure rules via a **.yamllint.yml** file in the repository root. For example, to allow longer lines:

```
extends: default
rules:
  line-length:
    max: 120
  indentation:
    spaces: 2
    indent-sequences: consistent
```

7.4 Write a Kubernetes Pod YAML (Simplified Preview) To further practise, write a minimal Kubernetes Pod definition. This will be fully explored in later sessions, but it introduces the YAML structure of a real-world manifest.

```
# mern-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: mern-backend
  labels:
    app: mern
    tier: backend
spec:
  containers:
    - name: backend-container
      image: your-username/mern-backend:1.0.0
      ports:
        - containerPort: 5000
      env:
        - name: NODE_ENV
          value: "production"
        - name: DB_HOST
          value: "mysql"
      volumeMounts:
        - mountPath: /app/uploads
          name: upload-storage
  volumes:
    - name: upload-storage
      emptyDir: {}
```

Validate with **yamllint mern-pod.yaml**.

Discuss the structure:

- Mappings: **metadata**, **spec**.
- Lists: **containers**, **ports**, **env**.
- List items are mappings with their own keys.

7.5 Experiment with Anchors

Create a file `anchors-demo.yaml`:

```
# anchors-demo.yaml
common_env: &common
  NODE_ENV: production
  LOG_LEVEL: info

services:
  backend:
    image: mern-backend:1.0.0
    environment:
      <<: *common
      PORT: "5000"
  frontend:
    image: mern-frontend:1.0.0
    environment:
      <<: *common
      PORT: "80"
```

Yamllint will parse it correctly, and the merge key will be validated.

8. YAML in the DevOps Toolchain (Context)

- **Docker Compose** (`docker-compose.yaml`): defines multi-container applications.
- **Kubernetes manifests**: all resource definitions (Deployment, Service, ConfigMap) are YAML.
- **Ansible playbooks**: tasks and inventories are YAML.
- **CI/CD**: GitHub Actions workflows, GitLab CI `.gitlab-ci.yml`, and CircleCI config are all YAML.
- **CloudFormation** (with YAML option), **Serverless Framework**, **Helm values** files.

Understanding YAML deeply prevents subtle errors (like a boolean `yes` being interpreted as `true`) that can cause production incidents.

9. Summary

- YAML is a human-readable data serialisation format that supersedes JSON and is ubiquitous in DevOps configuration.
- It uses indentation for structure, prohibits tabs, and supports scalars, sequences, and mappings in both block and flow styles.
- Multi-line strings can be literal (`|`) or folded (`>`); anchors and aliases reduce duplication.
- Multi-document files, separated by `---`, allow multiple YAML documents in a single file.
- The practical exercise created a container deployment description and a Kubernetes Pod YAML, and validated them with `yamllint`.
- Proficiency in YAML is essential for the DevOps i.e. Docker Compose, Kubernetes, and CI/CD pipeline definitions.

Session 8: Docker Compose – Defining and Running Multi-Container Applications

1. Overview

So far, we have managed individual containers using `docker run` commands and built images with Dockerfiles. In a real application, services rarely live alone. A MERN stack needs a database, a backend API server, and a frontend web server, each running in its own container, sharing networks, and starting in the correct order. Docker Compose is the tool that defines all of this in a single declarative YAML file and manages the full stack with a few commands. This session covers the Compose file structure, all essential directives, the `.env` file mechanism, health-aware startup ordering, and networking and volume management.

2. Learning Objectives

- Explain Docker Compose's role in multi-container application management and how it differs from plain `docker run`.
- Describe the evolution from legacy Compose v1 (`docker-compose` with hyphen, Python) to modern Compose v2/v5 (`docker compose` as a CLI plugin, Go, Compose Specification).
- Write a Compose file using the modern, version-less Compose Specification format.
- Define services using the `image` and `build` directives, map ports, and declare volumes and bind mounts.
- Use `depends_on` with `condition: service_healthy` to enforce correct startup ordering, and write health checks for MySQL and Node.js services.
- Manage environment variables through inline definitions and `.env` files, and understand variable precedence.
- Structure custom networks for service isolation and explain the default bridge network Compose creates.
- Issue essential Compose CLI commands to build, start, stop, inspect, and tear down the stack.

3. What Is Docker Compose?

Docker Compose is a tool for defining and running multi-container Docker applications. Using a YAML file, you configure all services, networks, and volumes that make up your application. With a single command, Compose creates and starts every container according to the declared specification.

Without Compose, launching the MERN stack would require three separate `docker run` commands with carefully matched network settings, volume flags, environment variables, and port mappings. With Compose, one file captures all of this logic, making the stack reproducible, version-controlled, and shareable.

Compose v1 vs v2/v5 (as of 2026)

Aspect	Compose v1 (legacy)	Compose v2 / v5 (modern)
CLI invocation	<code>docker-compose</code> (hyphen)	<code>docker compose</code> (space)
Implementation	Python stand-alone binary	Go plugin inside Docker CLI
File version field	Required: <code>version: '3.8'</code>	Obsolete and ignored
Specification	Pinned file formats (2.x, 3.x)	Rolling Compose Specification

Aspect	Compose v1 (legacy)	Compose v2 / v5 (modern)
SDK	None	Official Go SDK (v5)

The current versions are Docker Engine 29.x and Docker Compose plugin v5.1.x. The **version** top-level element is deprecated and silently ignored by Compose v2/v5. All file format versions (2.x, 3.x) were merged into the Compose Specification, which is forward-rolling and does not require—and discourages—a **version** key.

4. Compose File Structure

A modern Compose file has three main top-level elements:

```
services: # mandatory – all containers
networks: # optional – custom networks
volumes: # optional – named volumes
```

File Naming Conventions Compose looks for files in this order:

1. **compose.yaml** (preferred, introduced in 2024)
2. **compose.yml**
3. **docker-compose.yaml**
4. **docker-compose.yml**

All are accepted; any avoids the deprecation notice that now appears when using the legacy **docker-compose.yml** name without a **version** field.

5. Services – The Core Building Block

Each service represents a container. The most important directives are:

5.1 image vs build

```
services:
  cache:
    image: redis:7-alpine # pull from registry
  api:
    build: # build from Dockerfile
      context: ./backend
      dockerfile: Dockerfile
```

Use **image** for pre-built or third-party containers. Use **build** when you need a custom image from your source code. Compose builds tagged images automatically on **docker compose up** if a build section exists, or you can force a rebuild with **--build**.

5.2 ports

```
ports:
  - "8080:80" # host:container
  - "443:443"
  - "127.0.0.1:3306:3306" # bind to loopback only
```

Format is `HOST:CONTAINER`. Binding to `127.0.0.1` restricts access to the host machine, which is a security best practice for databases that should not be exposed to the network.

5.3 volumes

```
services:
  db:
    volumes:
      - mysqldata:/var/lib/mysql # named volume
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql # bind mount

volumes:
  mysqldata: # declare named volume at top level
```

Named volumes persist across `docker compose down`. They are only removed with `docker compose down -v`. Bind mounts connect a host path directly into the container—ideal for configuration files and development.

5.4 environment and env_file

Inline environment variables:

```
environment:
  NODE_ENV: production
  DB_HOST: mysql
```

Loading from an external file:

```
env_file:
  - .env
```

The `.env` file lives in the project root and contains key-value pairs:

```
MYSQL_ROOT_PASSWORD=securepass123
MYSQL_DATABASE=mern_app
```

Variables in the Compose file can reference `.env` values with `${VARIABLE}` syntax. Compose automatically loads variables from `.env` if the file exists.

5.5 depends_on with Health Checks The `depends_on` directive alone only controls container startup order—it does not wait for the dependent service to be ready to accept connections. This causes a classic race condition: the backend container starts before MySQL has finished initialising its data directory, resulting in connection failures.

To solve this, combine `depends_on` with `condition: service_healthy`:

```
services:
  backend:
    depends_on:
      mysql:
        condition: service_healthy # wait until health check passes

  mysql:
    image: mysql:8.4
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 40s
```

The `condition` can be:

- `service_started` — default; container has started but may not be ready.
- `service_healthy` — the health check command returns exit code 0.
- `service_completed_successfully` — for one-shot containers that exit 0.

Without `condition: service_healthy`, a service that defines a health check gets no special waiting behaviour—the dependency still only waits for the container to start.

5.6 restart Policy

```
restart: on-failure      # restart only on non-zero exit
restart: no              # default
restart: unless-stopped  # recommended for production - restart unless
                        stopped manually
restart: always          # restart always irrespective of success, failed,
                        or manual stop
```

`unless-stopped` restarts the container if it crashes or if the Docker daemon restarts, but respects a manual `docker compose stop`.

5.7 Resource Limits

```
deploy:
  resources:
    limits:
```

```
cpus: "2.0"
memory: 512M
reservations:
  cpus: "0.5"
  memory: 256M
```

Limits are hard caps. Reservations guarantee minimum allocations. These prevent a single container from starving others on shared hosts.

6. Networks

Compose creates a default bridge network for each project. All services can reach each other by service name —Compose runs an embedded DNS server that resolves service names to container IP addresses. This means backend code connects to `mysql` (the service name), not `localhost`.

Custom networks provide isolation between groups of services:

```
services:
  frontend:
    networks:
      - frontend-net
  backend:
    networks:
      - frontend-net
      - backend-net
  mysql:
    networks:
      - backend-net

networks:
  frontend-net:
  backend-net:
```

Here, the frontend cannot reach MySQL directly; it must go through the backend. This limits the blast radius of a compromised web container.

7. `.env` Files and Variable Substitution

Compose automatically reads variables from a `.env` file in the project directory. Variables are referenced in the Compose file with `${VAR}` or `${VAR:-default}` syntax.

```
# .env
MYSQL_ROOT_PASSWORD=prodsecret789
MYSQL_DATABASE=mern_app
MYSQL_USER=mern_user
```

```
MYSQL_PASSWORD=mern_pass
BACKEND_PORT=5000
```

```
# compose.yaml
services:
  mysql:
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
```

Verify resolved values with:

```
docker compose config
```

This command prints the fully interpolated YAML—invaluable for debugging missing or incorrect variables.

Security Note: Add `.env` to `.gitignore` and commit a `.env.example` with placeholder values so other developers know which variables to set without exposing secrets.

8. Essential Compose CLI Commands

Command	Purpose
<code>docker compose up -d</code>	Start all services in detached mode
<code>docker compose up --build</code>	Rebuild images and start services
<code>docker compose down</code>	Stop and remove containers and default network
<code>docker compose down -v</code>	Also remove named volumes (destructive)
<code>docker compose ps</code>	List services and their state
<code>docker compose logs -f</code>	Follow logs from all services
<code>docker compose logs backend</code>	Logs from a specific service
<code>docker compose exec backend sh</code>	Open a shell in a running service
<code>docker compose restart</code>	Restart all services
<code>docker compose config</code>	Validate and view resolved configuration
<code>docker compose build</code>	Build or rebuild images
<code>docker compose pull</code>	Pull latest image versions

9. Practical Lab – Compose for the MERN Stack

9.1 Project Context You will create a `compose.yaml` that defines three services:

- **mysql** — MySQL 8.4 with a named volume for data persistence.
- **backend** — Node.js/Express API server, built from the `backend/` Dockerfile.
- **frontend** — React app served by Nginx, built from the `frontend/` Dockerfile.

All services communicate on a shared network. The backend waits for MySQL to pass its health check before starting.

9.2 Prerequisites Ensure the project structure from earlier sessions exists:

```
cd ~/devops-course/mern-app
ls -la
# Should contain backend/, frontend/, .dockerignore
```

Verify the Dockerfiles exist:

```
cat backend/Dockerfile
cat frontend/Dockerfile
```

9.3 Create the .env File

```
cat <<EOF > .env
MYSQL_ROOT_PASSWORD=devrootpass
MYSQL_DATABASE=mern_app
MYSQL_USER=mern_user
MYSQL_PASSWORD=mern_userpass
EOF
```

Add `.env` to `.gitignore`:

```
echo ".env" >> .gitignore
```

9.4 Create .env.example

```
cat <<EOF > .env.example
MYSQL_ROOT_PASSWORD=change_me
MYSQL_DATABASE=mern_app
MYSQL_USER=mern_user
MYSQL_PASSWORD=change_me
EOF
```

9.5 Update Backend to Use MySQL

The backend needs to connect to MySQL using environment variables. Update `backend/server.js` to include a database connection:

```
const express = require("express");
const mysql = require("mysql2/promise");
const app = express();
const port = process.env.PORT || 5000;

let db;

async function initDB() {
  db = await mysql.createPool({
    host: process.env.DB_HOST || "mysql",
    user: process.env.DB_USER || "mern_user",
    password: process.env.DB_PASSWORD || "mern_userpass",
    database: process.env.DB_NAME || "mern_app",
    waitForConnections: true,
    connectionLimit: 10,
  });
}

app.get("/", (req, res) => {
  res.json({ message: "MERN Backend is running" });
});

app.get("/health", (req, res) => {
  res.status(200).json({ status: "healthy" });
});

app.get("/db-check", async (req, res) => {
  try {
    const [rows] = await db.query("SELECT 1 + 1 AS result");
    res.json({ db: "connected", result: rows[0].result });
  } catch (err) {
    res.status(500).json({ db: "error", message: err.message });
  }
});

async function start() {
  await initDB();
  app.listen(port, () => {
    console.log(`Server listening on port ${port}`);
  });
}

start();
```

Add the MySQL driver dependency:

```
cd ~/devops-course/mern-app/backend
npm install mysql2
```

9.6 Update Backend Dockerfile for Production Image The existing backend Dockerfile from Session 5 already uses a multi-stage build. Ensure it installs dependencies correctly for production:

```
# backend/Dockerfile
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force
COPY . .

FROM node:22-alpine
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY --from=builder /app ./
ENV NODE_ENV=production
ENV PORT=5000
EXPOSE 5000
USER appuser
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD node -e "require('http').get('http://localhost:5000/health', (r) =>
    {process.exit(r.statusCode === 200 ? 0 : 1)})"
CMD ["node", "server.js"]
```

9.7 Create the Compose File Create `compose.yaml` in the project root:

```
# compose.yaml
# MERN Stack - MySQL + Node.js/Express + React/Nginx

services:
  mysql:
    image: mysql:8.4
    container_name: mern-mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    ports:
      - "127.0.0.1:3306:3306"
    volumes:
      - mysqldata:/var/lib/mysql
    networks:
      - backend-net
    healthcheck:
```

```
test:
  [
    "CMD",
    "mysqladmin",
    "ping",
    "-h",
    "localhost",
    "-u",
    "root",
    "-p${MYSQL_ROOT_PASSWORD}",
  ]
interval: 10s
timeout: 5s
retries: 5
start_period: 40s

backend:
  build:
    context: .
    dockerfile: backend/Dockerfile
  container_name: mern-backend
  restart: unless-stopped
  ports:
    - "${BACKEND_PORT:-5000}:5000"
  environment:
    NODE_ENV: production
    DB_HOST: mysql
    DB_USER: ${MYSQL_USER}
    DB_PASSWORD: ${MYSQL_PASSWORD}
    DB_NAME: ${MYSQL_DATABASE}
  depends_on:
    mysql:
      condition: service_healthy
  networks:
    - backend-net
    - frontend-net

frontend:
  build:
    context: .
    dockerfile: frontend/Dockerfile
  container_name: mern-frontend
  restart: unless-stopped
  ports:
    - "${FRONTEND_PORT:-80}:80"
  depends_on:
    - backend
  networks:
    - frontend-net

volumes:
  mysqldata:
    driver: local
```

```
networks:  
  frontend-net:  
  backend-net:
```

Design Decisions in This Compose File:

- The `version` key is omitted, as required by the Compose Specification for v2/v5.
- MySQL port `3306` is bound to `127.0.0.1` so it is reachable only from the host, not the network. Services inside Compose reach MySQL via the `mysql` service name on the `backend-net` network.
- The `backend` service uses `condition: service_healthy` on MySQL, so it waits for `mysqladmin ping` to succeed before starting. The `start_period: 40s` accounts for MySQL's initial data directory initialisation.
- Two networks isolate the frontend from the database directly: `frontend` can only communicate with `backend`; `backend` can communicate with `mysql`. This is a defence-in-depth pattern.
- Ports for backend and frontend use `${VAR:-default}` syntax so they can be overridden via `.env` or shell environment.
- All services use `restart: unless-stopped`, which survives both crashes and Docker daemon restarts but respects intentional shutdowns.

9.8 Validate the Compose File

```
cd ~/devops-course/mern-app  
docker compose config
```

This outputs the fully resolved YAML with all variables substituted. Check that passwords appear correctly and that the health check command includes the actual password value.

9.9 Start the Full Stack

```
docker compose up -d
```

Compose will:

1. Create the named volume `mern-app_mysqldata` and the two custom networks.
2. Pull the `mysql:8.4` image.
3. Build the backend and frontend images from their Dockerfiles.
4. Start the MySQL container and wait for its health check to pass.
5. Start the backend container once MySQL is healthy.
6. Start the frontend container after the backend.

Observe the dependency ordering in the output. If MySQL takes longer than 40 seconds to initialise, the health check will fail and Compose will not start the backend.

9.10 Verify the Stack

```
docker compose ps
```

Expected output shows three services, all with State **Up** and healthy.

Test the backend:

```
curl http://localhost:5000/  
curl http://localhost:5000/health  
curl http://localhost:5000/db-check
```

The **/db-check** endpoint confirms the backend can query MySQL through the internal network.

Test the frontend:

```
curl -I http://localhost:80/
```

Open a browser to **http://<server-ip>** to see the React application.

9.11 Inspect Logs

```
docker compose logs mysql      # MySQL startup and health check logs  
docker compose logs backend    # see DB connection confirmation  
docker compose logs -f         # follow all logs in real time
```

9.12 Shell into a Running Service

```
docker compose exec backend sh  
# Inside the container:  
env | sort  
node -e "console.log('test')"  
exit
```

9.13 Tear Down

```
docker compose down
```

This stops and removes containers and the default and custom networks, but keeps the named volume (**mysqldata**). To also remove the volume:

```
docker compose down -v
```

Use `-v` cautiously—it permanently deletes all data stored in named volumes.

9.14 Rebuild After Code Changes

If you modify backend or frontend source code:

```
docker compose up -d --build
```

The `--build` flag forces a rebuild of images before recreating containers.

10. Summary

- Docker Compose defines multi-container applications in a single, declarative YAML file, replacing multiple `docker run` commands.
 - As of 2026, Compose is a CLI plugin invoked as `docker compose`. The `version` field is obsolete. The Compose Specification is the authoritative format.
 - The three top-level elements are `services`, `networks`, and `volumes`. Services use `image` for pre-built containers or `build` for custom Dockerfiles.
 - `depends_on` with `condition: service_healthy` and properly defined `healthcheck` blocks eliminate startup race conditions between dependent services.
 - Environment variables can be defined inline, loaded from an `.env` file, and interpolated with `${VAR}` syntax. Use `docker compose config` to verify resolution.
 - Custom networks isolate service groups; the built-in DNS resolves service names to IP addresses automatically.
 - Key commands: `docker compose up -d`, `docker compose down`, `docker compose ps`, `docker compose logs`, `docker compose exec`, and `docker compose config`.
 - The practical lab created a complete `compose.yaml` for the MERN stack—MySQL, Node.js backend, Nginx-served React frontend—with health checks, secrets via `.env`, and network segmentation.
-